Making a pinball game for Playdate: Part 01, the language

Sunday, December 1, 2024

Source: https://amano.games/devlog/making-a-pinball-game-for-the-playdate-part-01-the-language

Welcome to this December adventure, where I will try to write about the process of our latest game, Devils on the Moon pinball.

If you want to see the game in action, you can watch the Playdate Fall update where we show a sneak peek of the game.

After we released Pullfrog Deluxe I had mixed feelings. On one side, I was super happy with the game that we did, but on the other, I felt stuck. There are some parts of Pullfrog that I felt I could have done better. I have had this feeling before; the problem is that I didn't know how.

I'm not a person who cares a lot about which programming language he uses; I have some preferences, but mainly because of familiarity. I have used *JS* for 10 years, almost daily, so whenever I have to make something fast, I tend to pick that. But the last time I programmed in a low-level language like *C* or *C++* was at university, more than 10 years ago.

When we were in the last stretch of **Pullfrog's** development, one thing that would happen often is that we didn't know if we could add something to the game without running in to performance issues, for example we had more ideas for special pieces but any time we added a little visual effects we would spend days trying to optimize the game and make it run at **30FPS**.

In the current version, at the end of 2024, the game still *struggles* when you have a lot of pieces in the board all moving and falling. No one has ever complained about it. I think most people assume we slow down the game on purpose to make it more *dramatic* or easier to read, similar to how space invaders would speed up the fewer enemies were on-screen. But truth be told, I was annoyed by it, but it was good enough.

In the process of trying to optimize *Pullfrog* I rewrote large chunks of the game and underlying systems, some obvious things had great results, but by the end I reached my capacity to improve the codebase. I realized that even with more time, I didn't know how to make the code better.

You always have constraints when doing a game; there are fun ones, like a limited color palette, but in this case I couldn't shake the feeling that we were missing on some good game design ideas by being afraid of performance.

I always have had a curiosity about low-level languages, game engines and, as the name of our studio says, doing things from scratch. So one year ago, I started my last December adventure; I started the Hand Made Hero video series and followed along. This was my second attempt at it. The first time I tried it was back when the project launched, but I quickly gave up after trying to set up a dev environment on Linux and translating the platform code from Windows to Linux.

This time I thought there were a couple of good resources that helped me get over the hump. I made it up to day 32; it was great!. I have tried a couple of times to make games in *C*, mainly following small tutorials, and it just felt like doing games in any other language/framework but worse. Watching Casey make something from nothing and understanding most of it felt empowering, and kind of scary.

I realized that there were a ton of things to learn while doing games this way, and maybe that would help me to become better at doing them. So by the start of the next year I made a decision; I was going to make a game in *C* and tried to do everything myself.

The main rule I set for myself was that I prefer to make something simple and do it myself, rather than trying to do something complex that I don't understand and grabbing someone else's code.

Spoiler alert: I ended up doing the most complicated game mechanics I have done in the last 10 years. And I had a *great* time doing it.

See you in the next adventure, Part 02: The Physics.